

Starship heat tile traceability

Preliminary design review. Yifan Guo. Sept 23, 2026.

1. Problem

What is broken for ops/quality is that genealogy is not recorded as lots flow through the line. As new materials move through the system, they are incorporated into a mixture and move in proportion with other elements of the mix.

We cannot apply a start/stop-per-unit model because supplies are added on a continuous time scale, and a vessel is never emptied. We are constantly redefining what a product is.

A serial number exists only when a tower casts a tile. A serialized tile must contain the purchased receipts (FIB-001, ACE-001, NI-001, ...). Genealogy is parent lots → mass (kg), written at TILE_STAMP.

2. Goals and requirements

Outcome

Establish a robust and digestible genealogy of every produced tile. The report should be transparent, containing the exact mass of each lot. “This tile contains lot X” means lot X contributed a computed mass to that 8 kg slice under the allocation policy below.

Must haves

- What is in the genealogy report for a tile, and how to interpret it: lotId + kg per parent lot.
- A lot allocation policy to track how lots flow through the system and when they are cast.
- What gets written to the ledger, inventory, and MES state

Later

- How to deal with crashes / restarts. Out of scope to implement in this PDR, but I did attempt one.
- When to drop a lot entirely (threshold). Default below.

Disagreeing meters, late scans, and taken-down vessels have defaults in §3 and §5. They are not deferred.

Scale

The spec is ~500 tiles/day on two towers. Each tile is 8 kg. Tanks 100–400 kg, towers 80 kg full. Lots are added in much smaller amounts (gas even smaller). Mid-volume event traffic does not necessitate a scheduling layer. MES accepts near-simultaneous adds when more than one ingredient is low - each operation should be quick and lightweight.

3. Proposed design

Lot allocation

The lot allocation policy dictates how mass moves through the system. For simplicity I opted for an instant-mixing model. Two of the ingredients are liquids, easy to incorporate. The nickel is sharded so it can be incorporated too. This removes the need to use calculus to determine transfer rates as parts move through the system. Instead we estimate the mass of a lot leaving a vessel as a proportion of how much of that lot was in the vessel at the time:

$$mass_out_for_lot = (current_mass_lot / total_mass_vessel) \times total_mass_out \quad (D1)$$

MES and inventory

The MES is the living system of record of lot composition as it flows through the line. For each vessel it tracks how much mass of which lot is in it, using live tags from the PLC and the allocation policy. That map is lotId → kg. If possible, the MES should be a distributed fleet of servers (rather than a central one), each acting as a gRPC client. If one server crashes, other vessels do not lose their state and can take over production immediately. Since the PLC provides readings per second, requests from MES to PLC must be lightweight and fast. Prefer gRPC for latency-optimized, strict machine-to-machine communication between MES and PLC.

A critical function of the MES is the ability to rebuild its state on restart - an untraceable mass can corrupt downstream vessels as it flows through the system. We cannot rebuild the state from the ledger (plus current total mass) only because the MES does not know what was in the system at the time of the crash. The system must therefore restore state from its most recent backup and replay events between the snapshot time until now. *See appendix for an example of a replay operation.*

To accomplish replay, the MES must backup its current state periodically. The point-in-time snapshot should contain (1) key-value pair of lotId → mass (kg) and (2) a timestamp of when the snapshot was produced. The backup interval should be frequent but not every second - it depends on how often the composition is updated (1/s flow readings) and how often replays occur. Backups should also be performed by a sidecar to decouple persistence from the MES critical path - the per second composition update.

Inventory records availability of purchased lots on reserve. A transfer inside the line does not touch inventory. The warehouse lots go down on user add operations. Since multiple batches of the same lot can arrive in the warehouse, a separate table serves the purpose of tracking inventory totals per lot, while a separate table tracks individual purchase orders. *See appendix for a proposed data model for inventory.*

An API contract handles purchase orders and persists records to the inventory database. The system must favor correctness over speed - a write cannot succeed unless the backing store is updated. Prefer the API to use REST since the call volume is not as frequent as the meter refresh intervals and REST integrates more readily with user interfaces for simplified inventory management.

Add

On an add, a specified mass is introduced into the system. The mass and lot (if specified) is deducted from inventory, written to the ledger, and added to the MES vessel map. If the incoming mass is a transfer from another vessel, use the allocation policy against the source vessel's map. If we cannot determine composition (unattributed raw add), record a new key UNATTRIBUTED-n, instead of pooling unmatched adds into one bucket. A later scan appends scan-attach to the ledger and renames that key on live maps (O(1) lookup on each vessel) and on already-cast tiles that still hold the placeholder (select with filter).

Consume — flow and cast

On a consume, we record the type of action in the ledger (set flow from → to kg/min, stop flow, or cast + 8 kg slice). Vessel composition maps update at every sample interval (using the lightweight gRPC calls to the PLC) from measured Δkg. For a cast, the exit mass is 8 kg. MES walks every lot on the tower map and applies equation D1 above. Those amounts come off the tower map and become tile_lot rows on the child unit.

For a flow, the same split applies to the measured mass that left the vessel. Since level also moves on add, it is an unreliable outflow signal by itself. Prefer the flow meter when one exists (TANK-A outlet, TOWER-1 inlet).

Flow math

$$kg_moved \approx mean(outlet_lpm) \times minutes \times 1.15 \text{ (slurry density 1.15 kg/L)}$$

Level outflow is net level-mass change minus known adds introduced in this interval. TANK-B and TOWER-2 have no flow meter, so we must use level readings to update these vessels.

$$(Level_pct - level_pct_{t-1}) / 100 \times full_kg - mass_added_this_tick \text{ \# backup if flow meter is not available}$$

On a cast, the mass to deduct from each lot in the map is proportioned from the fixed 8kg. A set of records is written to the tile which contain the measured mass for each lot in the tower at the time of cast.

What each op writes

Op	Ledger	Inventory	MES map
User add, lot scanned	append add (lot, kg, vessel, time)	warehouse lot down	that lotId += kg on the vessel
User add, no lot yet	append add UNATTRIBUTED-n	warehouse down if the receipt is known	placeholder key += kg. Do not pool.
Set flow from → to kg/min	append setpoint only	unchanged	no write until measured Δkg
Measured mass moved	no 1s row. Map updates on the sample.	unchanged (inside the line)	D1 slice off source, same slice onto dest
Stop flow	append stop (observed, not a commanded empty)	unchanged	no further consume on that path
Cast 8 kg + TILE_STAMP	append cast + 8 kg slice	unchanged	D1 8 kg off tower. Write tile_lot rows.
Late scan	append scan-attach. Do not edit the add.	unchanged	rename that key on maps and on tiles that held it
Taken-down	append observed stop	unchanged	freeze that vessel map. Lots stay. MES crash ≠ taken-down.

Genealogy is read from the vessel map at cast, not reconstructed by walking every 1 sec / row.

A fiber lot from TANK-A through a tile that also has nickel

Time	Event	TANK-A map (kg)	TOWER-1 map (kg)
08:00	Add FIB-001 40 kg to A. Warehouse FIB-001 down 40.	FIB 40 (40)	—
08:02	Add ACE-001 15 kg to A.	FIB 40, ACE 15 (55)	—
08:04	Set A → T1 at 3 kg/min. Ledger records the setpoint only.	55, flowing	starts receiving D1 slices
08:04–08:10	Measured 18 kg A → T1 (not 3×6 assumed from the setpoint).	FIB 26.91, ACE 10.09 (37)	FIB 13.09, ACE 4.91 (18)
08:10	Add NI-001 2 kg to T1.	37	FIB 13.09, ACE 4.91, NI 2.00 (20)
08:11	Add HP-001 5 kg to T1.	37	... + HP 5.00 (25)
08:14	Set A flow to 0. Apply D1 on each sample 08:10–08:14, then stop.	frozen after last Δkg	holds fiber + acid + nickel + peroxide
08:22	Cast 8 kg from T1. TILE_STAMP T-104.	unchanged	minus the 8 kg slice

At cast, walk every lot on TOWER-1. If the tower holds 25.7 kg (FIB-001 14.0, ACE-001 5.25, NI-001 1.85, HP-001 4.60):

$$qty_FIB-001_in_tile = 14.0 / 25.7 \times 8 \approx 4.36 \text{ kg}$$

Same fraction for ACE, NI, HP. Tile T-104 therefore contains fiber lot FIB-001 and nickel lot NI-001. Write four T-104 / TOWER-1 / 08:22 tile_lot rows - one for each lot. That is the genealogy report.

TANK-B, taken-down, late scan

TANK-B has no outlet meter. Infer what left it from the per-sample diff of level-mass, minus known adds, then run the same map. The 1s loop keeps the last PLC sample in memory on that vessel (last_level_pct).

When a tank or tower is taken down mid-run (out of service), we freeze that vessel map. Lots stay inside. Record OOS and flow stop on the ledger.

On a late scan, any unattributed mass already lives in the vessel map, the scan just provides the lotId. Since we keep separate UNATTRIBUTED-n keys for each unidentified setpoint, we can replace the placeholder with the lot. Append scan-attach to the ledger. Rename the key on live maps and on tiles that still list that placeholder.

When to remove a lot from a vessel's map?

Since vessels never fully empty and the lot mass approaches but never converges to zero, lots never fully disappear, which causes unbounded memory growth. To clean up trivial-trace lots from the system, we set a threshold mass of During each vessel's update cycle, it checks and removes lots below this threshold from its map. To come up with this threshold, suppose we have a 100kg mass where the *least* common lot (NIC) is only 2kg. Assuming a standard 3kg removal rate of the remaining total mass per minute while a third lot is introduced to keep the total mass identical, the NIC lot retains about 22% (0.44kg) of its mass after 50 iterations - a reasonable threshold for this problem. This recommended the initial threshold value of 0.5kg.

Tank or Tower crashes, restore & Replay

On an MES crash, the pumps keep running.

If a tank or tower crashes unexpectedly, and is brought back online - we need to restore the vessel map. Read the latest snapshot for that vessel. Determine the current mass from PCL.level_pct and calculate the mass difference from the time of last snapshot, subtracting any adds & casts on the ledger since the time of crash. If there is a net increase in total mass, add that mass as UNATTRIBUTED to the snapshot, since we have no way of knowing where that additional mass came from. If there is a net decrease, adjust each lot in the snapshot by its weighted proportion. Finally, apply the user-added lots onto the map. The state is now ready for commission. *See algorithm and chart below for a walkthrough of Replay.*

Algorithm:

Read current mass from the PLC:

$$M_{\text{now}} = (\text{level pct} / 100) \times \text{full kg}$$

From the ledger after the snapshot, total the logged adds A and casts C.

Treat unlogged pumping as happening before the adds. Mass of the old mix after that hole:

$$M_{\text{old}} = M_{\text{now}} - A + C$$

- If $M_{\text{old}} > M_s$: keep the snapshot lots. Extra mass ($M_{\text{old}} - M_s$) is UNATTRIBUTED.
- If $M_{\text{old}} < M_s$: scale every snapshot lot by (M_{old} / M_s).
- If $M_{\text{old}} < 0$: stop (invalid). The radar does not contain the logged adds.

Then take the logged casts off the map in the same ratios, then put each logged add on its lot.

The map now totals M_{now} . UNATTRIBUTED appears only when the radar is heavier than snapshot + adds – casts can explain.

On Replay (No Adds since Crash)

Time	Event	TANK-A map (kg)	TOWER-1 map (kg)
08:00	Add FIB-001 70 kg to A. Warehouse FIB-001 down 40.	FIB-001 (70)	—

08:01	Add ACE-001 30 kg to A. Warehouse ACE-001 down 30	FIB-001 (70) ACE-001 (30) *snapshot(total_mass=100kg)	—
08:04	Start Flow A->T1. Setpoint 3 kg/min		—
08:09	MES Tank-A crashes	FIB-001 (59.5) ACE-001 (25.5)	FIB-001 (10.5) ACE-001 (4.5) * true values derive from readings
08:10	MES Tank-A restored	*restore from snapshot FIB-001 (70) ACE-001 (30) PCL level reading (84kg) 100 - 84 = 16kg No adds in ledger FIB-001 = 16 * 7/10 = 11.2 kg ACE-001 = 16 * 3/10 = 4.8 kg FInal State: FIB-001 (59.8) ACE-001 (25.2)	FIB-001 (11.0) ACE-001 (5.0) * true values derive from readings

4. Alternative designs

Decided not to go with a constant-time mixing model. That approach complicates the mass calculations and also doesn't account for the fact that most of the materials are readily incorporated (two liquids, and one solid in fragments). Even a linear incorporate-over-K-minutes model still has to answer how much of each lot exits while it is only partly mixed.

I also disregarded a FIFO/LIFO approach because we assume produced tiles require mixtures, rather than fixed building blocks. If we treat ingredients as discrete blocks that never mix, some elements might never land in a tile depending on where in the stage they were introduced. FIFO/LIFO is out, not a consume order sitting under the well-mix map.

Also decided against a queuing system for inventory because lot deductions must operate on the real-time state of the system, rather than wait for a callback. The MES should not assume an add operation completed if the transaction has not been processed from the queue, since other transactions may ask for an amount of the same lot that exceeds the amount that exists in inventory.

For the replay mechanism, I decided against relying on only the ledger and the location's current mass alone because the same total mass can occur multiple times along the history of the vessel, and the MES wouldn't know which one to land on. I also deferred making replay rely on coordination among multiple MES systems. While asking other tanks on every transfer would allow for a perfect map reconstruction across a crash, it tightly couples the four vessels on the critical update (1/s) path. The line does not need that. Better to have inbound mass be marked as UNATTRIBUTED than calling out to other computers.

5. Open questions

Track negligible amounts? When is a lot dropped from the map?	Default no unbounded dust. Drop a lot from a vessel map when it falls below a threshold (start at 0.01 kg). Do not delete it from the ledger. Ledger stays append-only.
TANK-B has no outlet meter — how do we infer what left it?	Per-sample level-mass diff minus known adds. Same D1 map as A.

--	--

Tank or tower taken down mid-run — lots stranded inside?	Mixture stays. Flow stops. Map freezes. Observed stop on the ledger. Other in-service vessels still run.
Lot scan arrives after mass has already moved downstream?	Separate UNATTRIBUTED-n per unmatched add. Later scan-attach renames that key on maps and on tiles that still list it.
Two sensors disagree on mass moved — which wins?	If the gap exceeds 5% or 2 kg, flag it. Then prefer the flow meter. B and T2 have no output flow meter so there is nothing to disagree on.
A transfer only has source location id, no lot — how does composition travel?	Answered by the allocation policy. D1 on the source map. Copy that slice onto the destination. Inventory unchanged.
Edit the ledger on late scan?	No. Ledger is append-only. Update in-memory maps.
JSON blob per tile?	No. tile_lot rows so scan-attach is an UPDATE on lot_id.
MES crash restore?	Use last saved maps + replay events forward. Location total mass cannot recover the split. No-snapshot history and mismatch with readings defaults to UNATTRIBUTED

PDR: this PDF. Decision notes: heat-tile-decisions.pptx. Line simulation (required, separate): heat-tile-line-simulator.html — each tile's parent lots and quantities under the instant well-mix split.

Appendix

Tile Genealogy

Tile serial number int	Non-unique identifier tracing the lot material to this specific tile
Sourced_location string	Location the tile was casted from
Lot_id uuid	Lot identifier used in the production of this tile
Mass float	Mass (kg) of the lot used in the production of this tile

Post-production Genealogy analysis

SELECT * from Tile_Genealogy WHERE tile_serial_number = 123; # genealogy of specific tile

SELECT * from Tile_Genealogy WHERE lot_id = X; # lineage of all tiles containing lot X

SELECT * from Tile_Genealogy WHERE sourced_location = 'LOC_TANK-A' AND lot_id = X AND mass >= 50; # genealogy of all tiles containing more than 50 kg of lot X

Late Scan Update

UPDATE Tile_Genealogy SET lot_id = FIB-293 WHERE lot_id = UNSPECIFIED_n;

Raw Material Inventory (add + update)

Lot_id uuid	Unique identifier of the purchased lot
Remaining_kg float	Total Mass (kg) of the lot in the warehouse

Purchase Transaction Table (append-only)

Lot_id uuid	Unique identifier of the purchased lot
Mass float	Mass (kg) of the lot added
Order_id uuid	Unique identifier of the order
Ordered_at timestamp	Time the order is placed

Update Raw_Material SET mass=mass - kg WHERE lot_id = :id AND mass >= :kg; # deduct from inventory on add

HTML simulator is a separate deliverable — not inside this PDR 4